

# UF-Protect – SCE Profile v1.0

## UF-Core / Structural Cryptography Engine

### 1 Purpose and Scope

This document specifies how the Structural Cryptography Engine (SCE) instantiates the External Crypto Provider required by the UF-Protect v1.1 UF-Core Kernel Security Specification.

It covers only:

- Key hierarchy and key derivation.
- UF-SP envelope encryption.
- Keying for dialects and decoy (“ink”).
- Custody logging hooks.

It does *not* define SCE internals (HSM layout, physical controls, etc.).

### 2 External Crypto Provider Interface

UF-Protect requires an External Crypto Provider with the following abstract interface:

- $\text{GetRootKey}(\text{context}) \rightarrow K_{\text{root}}$
- $\text{DeriveKey}(K_{\text{root}}, \text{label}, \text{domain}) \rightarrow K_{\text{domain}}$
- $\text{AEAD\_Encrypt}(K_{\text{domain}}, \text{aad}, M) \rightarrow (C, \text{tag})$
- $\text{AEAD\_Decrypt}(K_{\text{domain}}, \text{aad}, C, \text{tag}) \rightarrow M$  or error

SCE SHALL implement this interface as described below.

### 3 Key Hierarchy

#### 3.1 Root Keys

SCE maintains one or more long-lived root keys  $K_{\text{root}}$  inside its own trust boundary (e.g. HSM).

(K1) Root keys MUST never leave SCE in plaintext.

(K2) Each UF deployment profile (environment, region, or major version) MUST have a distinct root key identifier.

### 3.2 Domain Keys

Domain parameters  $D_{\text{dom}}$  include, at minimum:

- Environment (e.g. `dev`, `prod`).
- Region (e.g. `us-east-1`).
- Logical domain (e.g. `"uf-kernel"`, `"uf-sp"`).
- Optional device or tenant identifiers.

SCE derives domain keys using HKDF:

$$K_{\text{dom}} = \text{HKDF}(K_{\text{root}}, \text{label} \parallel D_{\text{dom}}).$$

The following labels MUST be reserved for UF-Core:

- `"UF-SP"` — for UF Structural Payload envelopes.
- `"UF-DIALECT"` — for UF-IL dialect keys.
- `"UF-DECOY"` — for decoy/ink keys.

## 4 UF-SP Envelopes with SCE

### 4.1 Encryption

Given UF-SP (structural payload) serialized as  $M$ , SCE SHALL:

- (E1) Compute  $K_{\text{SP}} = \text{HKDF}(K_{\text{root}}, \text{"UF-SP"} \parallel D_{\text{dom}})$ .
- (E2) Compute  $(C, \text{tag}) = \text{AEAD\_Encrypt}(K_{\text{SP}}, \text{aad}, M)$  where  $\text{aad} = \text{Header} \parallel \text{Summary}$ .
- (E3) Return an envelope object:

$$\text{Envelope} = (\text{Header}, \text{Summary}, C, \text{tag}).$$

### 4.2 Decryption

To recover UF-SP:

- (D1) Recompute  $K_{\text{SP}} = \text{HKDF}(K_{\text{root}}, \text{"UF-SP"} \parallel D_{\text{dom}})$  using the same domain parameters.
- (D2) Invoke  $\text{AEAD\_Decrypt}(K_{\text{SP}}, \text{aad}, C, \text{tag})$ .
- (D3) On failure, SCE MUST return an error and MUST NOT return any plaintext.

UF-Core MUST treat any decryption failure as a T5 (envelope/domain mismatch) event and MUST NOT use any structural data from the envelope.

## 5 Dialect and Decoy Keys via SCE

### 5.1 Dialect Keys

For each UF-IL dialect identifier  $D$  (which may encode environment, device identity, or version), SCE derives

$$K_{\text{dialect}} = \text{HKDF}(K_{\text{root}}, \text{"UF-DIALECT"} \parallel D).$$

UF-VM then uses  $K_{\text{dialect}}$  to generate the opcode permutation  $\pi_D$  and any other dialect-specific encoding required by UF-Protect v1.1.

### 5.2 Decoy (Ink) Keys

For decoy summary generation, SCE derives

$$K_{\text{decoy}} = \text{HKDF}(K_{\text{root}}, \text{"UF-DECOY"} \parallel D_{\text{dom}}).$$

UF-Core uses  $K_{\text{decoy}}$  only in the decoy function defined in the UF-Protect v1.1 spec to compute  $SUM_{\text{decoy}}$ .

## 6 Custody Logging

When SCE is used as UF-Core’s External Crypto Provider, it SHOULD offer an API to record security-relevant events, including:

- Envelope creation and decryption.
- Mode transitions (0,1,2) requested by UF-Core.
- Integrity check failures and attestation failures.

UF-Core MAY call such an API with:

`RecordEvent(event_type, domain, metadata),`

where `metadata` is redacted to avoid exposing UF-SP.

## 7 Conformance

An SCE deployment is UF-Protect SCE Profile v1.0 compliant if:

- (S1) It supports the External Crypto Provider interface described in Section 2.
- (S2) It implements key hierarchy and derivation rules of Sections 3 and 4.
- (S3) It correctly encrypts/decrypts UF-SP envelopes as in Section 4.
- (S4) It provides at least basic custody logging as in Section 5, or an equivalent mechanism agreed with UF-Core integrators.